

Security Audit

VIA Labs (Cross-Chain Infrastructure)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	15
Audit Resources	15
Dependencies	15
Severity Definitions	16
Status Definitions	17
Audit Findings	18
Centralisation	45
Conclusion	46
Our Methodology	47
Disclaimers	49
About Hashlock	50

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The VIA Labs team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

VIA Labs is a cross-chain infrastructure protocol that specializes in bridging and messaging solutions. Their flagship product focuses on USDC bridging and Proto-USD implementation across multiple blockchains. The protocol enables secure and efficient cross-chain communication while maintaining asset stability through their unique gas reimbursement mechanism.

Project Name: VIA Labs

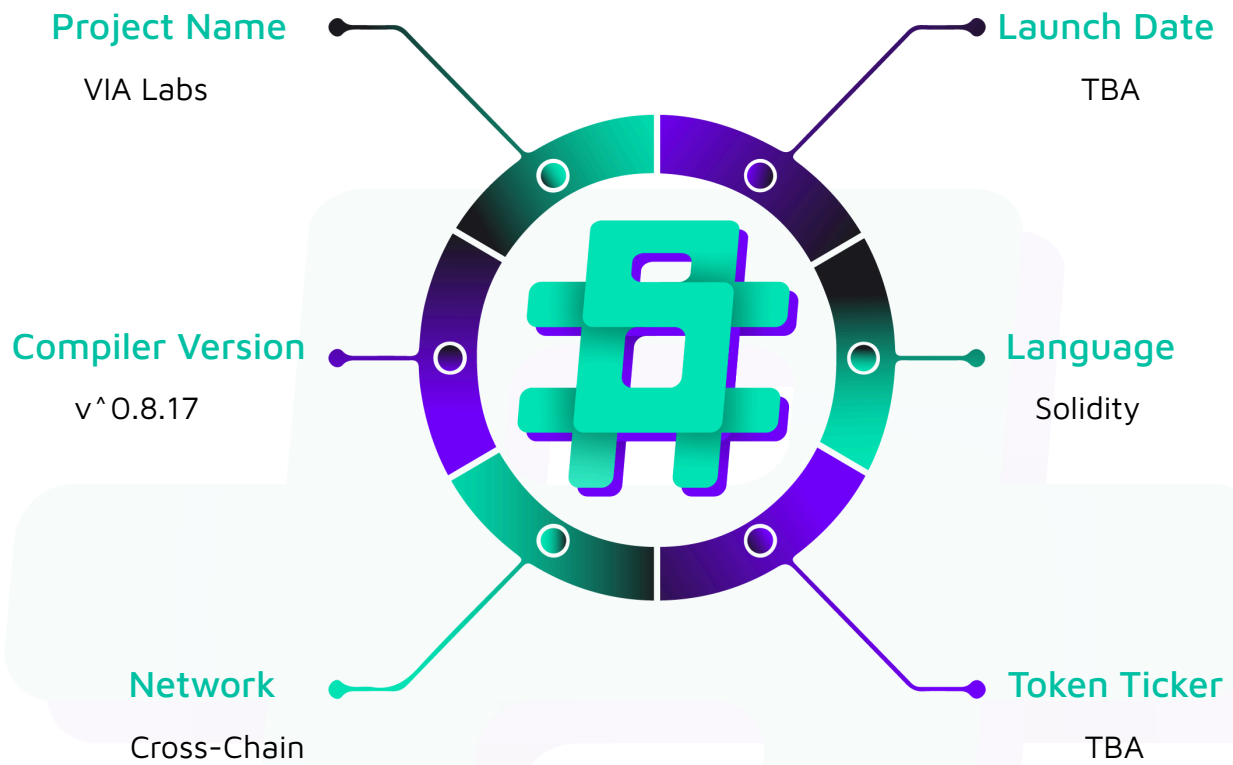
Project Type: Cross-Chain Infrastructure

Compiler Version: ^0.8.17

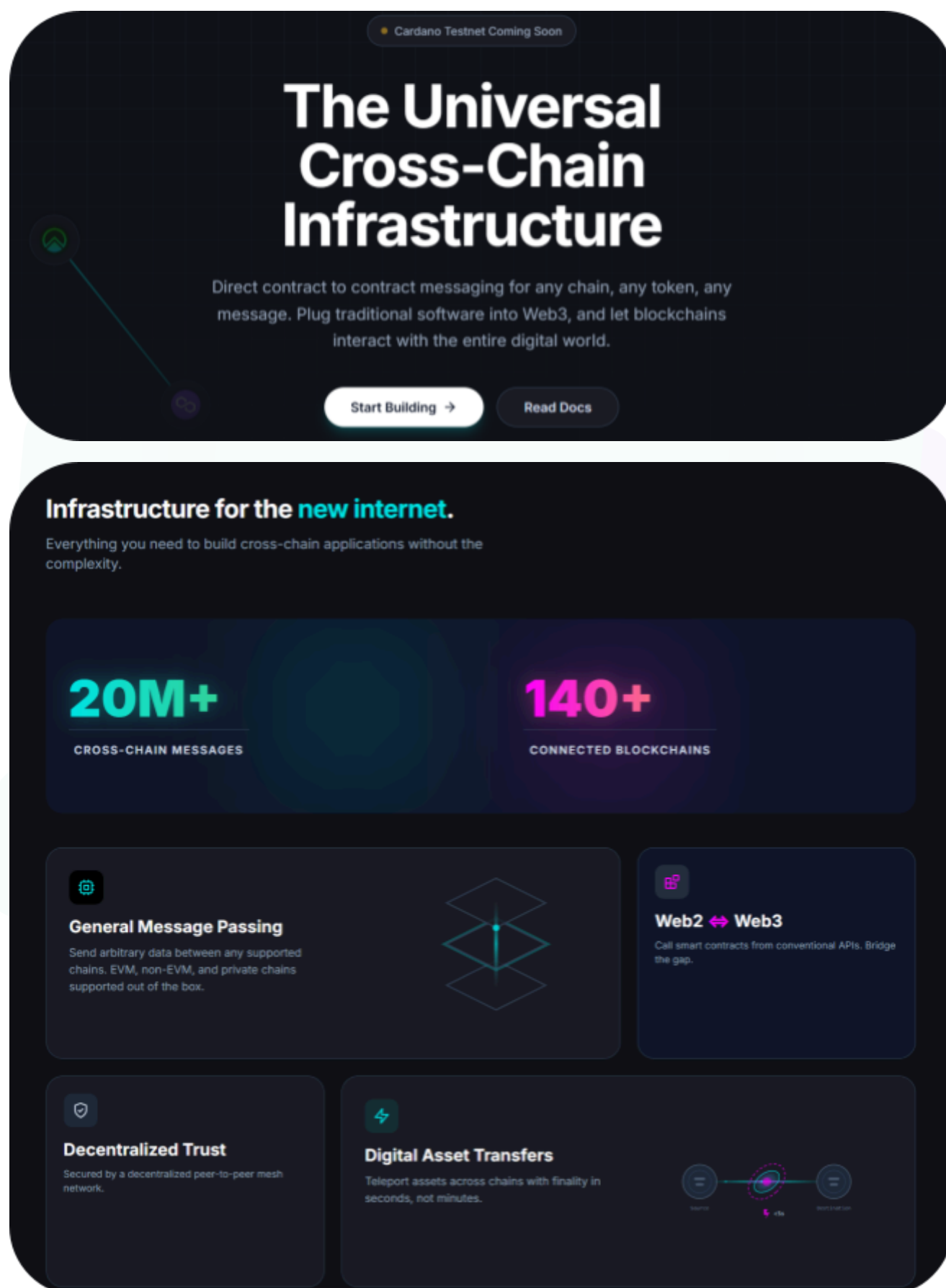
Website: <https://vialabs.tech>

Logo:



Visualised Context:

Project Visuals:



#hashlock.

Hashlock Pty Ltd

Audit Scope

We at Hashlock audited the solidity code within the VIA Labs project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	VIA Labs Smart Contracts
Network	Ethereum
Language	Solidity
Audit Date	June, 2026
Contract 1	BridgeManagerVG1.sol
Contract 2	IFiatToken.sol
Contract 3	IQuoterV2.sol
Contract 4	IUniswapV3Pool.sol
Contract 5	IUniversalRouter.sol
Contract 6	lib/uniswap/OracleLibrary.sol
Contract 7	lib/uniswap/FullMath.sol
Contract 8	lib/uniswap/TickMatch.sol
Audited GitHub Commit Hash	78e46597b569b6b66bf22932585ec119fcd99d5c
Fix Review GitHub Commit Hash	5f0e1bb1894cdff3ae9dff4575de51f9f5b208f2

Scope Note

This audit was limited to the contracts explicitly listed in the Audit Scope section. Contracts, libraries, integrations, gateways, relayers, offchain infrastructure, deployment configuration, and external protocol components not listed in this section were not subject to a full dedicated security review.

During the audit, Hashlock reviewed certain out of scope components only where they directly interacted with the in scope bridge flow. Any findings related to those components should be understood as limited observations and do not represent a complete assessment of those contracts or the wider VIA Labs ecosystem.

Hashlock recommends that any contracts or external components relied upon in production, including the gateway layer, be reviewed under a dedicated audit scope before continued reliance.

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

We initially identified some vulnerabilities that have since been addressed.

Hashlock found:

2 High severity vulnerabilities

3 Medium severity vulnerabilities

8 Low severity vulnerabilities

1 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
src/lib/uniswap/OracleLibrary.sol • A vendored Uniswap V3 periphery library that provides BridgeManagerVG1 with internal pricing helpers (no external/public surface; all functions are internal and reached only via delegation from the bridge): ◦ consult — reads the pool's observe() cumulative ticks and returns the time-weighted arithmeticMeanTick and harmonicMeanLiquidity over a secondsAgo window; rejects a zero window ("BP") and rounds the mean toward negative infinity. ◦ getQuoteAtTick — converts a tick plus a baseAmount into the equivalent quoteToken amount, branching on baseToken < quoteToken to handle token sort-order and on sqrtRatioX96 magnitude to preserve precision. ◦ getOldestObservationSecondsAgo — returns the age of the oldest stored observation (retained verbatim for upstream-diff	Contract achieves this functionality.

parity; not reached by the bridge).

- `getBlockStartingTickAndLiquidity` — returns the tick and liquidity as of the start of the current block (parity-only; not reached).
- `getWeightedArithmeticMeanTick` — computes a weight-averaged tick across pools (parity-only; not reached).
- `getChainedPrice` — composes a synthetic tick across a token route (parity-only; not reached).

src/lib/uniswap/FullMath.sol

- A vendored Uniswap V3 core library providing `BridgeManagerVG1` (via `OracleLibrary.getQuoteAtTick`) with full-precision 512-bit fixed-point math (all functions internal):
 - `mulDiv` — computes $\text{floor}(a * b / \text{denominator})$ with full 512-bit intermediate precision so phantom overflow of $a * b$ does not lose precision; reverts on `denominator == 0` or when the result would exceed `uint256`.
 - `mulDivRoundingUp` — the ceiling variant of `mulDiv` (retained verbatim for upstream-diff

Contract achieves this functionality.

parity; not reached by the bridge).	
src/lib/uniswap/TickMath.sol • A vendored Uniswap V3 core library providing BridgeManagerVG1 (via OracleLibrary.getQuoteAtTick) with tick ↔ sqrt-price conversion (all functions internal): ◦ getSqrtRatioAtTick — computes $\sqrt{1.0001^{\text{tick}} \cdot 2^{96}}$ as a Q64.96 number by binary-decomposing the tick and reverts "T" when tick exceeds MAX_TICK. ◦ getTickAtSqrtRatio — the inverse mapping (greatest tick whose ratio is $\leq$ the input), reverting "R" outside the representable range (retained verbatim for upstream-diff parity; not reached by the bridge).	Contract achieves this functionality.

src/BridgeManagerVG1.sol

- Allows the owner (setter / governance) to:
 - pause / unpause the bridge as the day-one Bridged-USDC hooks.
 - setMinBridgeAmount — set the minimum USDC amount accepted by bridge().
 - setBridgeConfirmations — set the block-confirmation hint forwarded to the gateway.
 - setSwapConfig — set the Universal Router, WETH, and pool fee tier used by swapGas.
 - setSwapPool — set the Uniswap V3 pool whose TWAP prices the WETH→USDC gas reimbursement.
 - setSwapSlippageBps — set the slippage multiplier ($\geq$ BPS_DIVISOR) applied to the TWAP quote.
 - setTwapPeriod — set the TWAP window (seconds) used to price gas refunds.
 - setSwapEnabled — toggle the dynamic-quote / auto-swap flow, realising any accrued reimbursements to the owner when flipped off.
 - withdrawWeth — withdraw WETH from the operational treasury.

Contract partially achieves this functionality.

- Authorise UUPS upgrades via `_authorizeUpgrade` (owner-gated).
- Allows the owner or projectOwner to:
 - `setProjectFee` — configure the optional per-bridge() fee (native or ERC-20), draining any old-token accrual on a token switch.
 - `withdrawProjectFees` — drain accrued project fees in the current fee token to a recipient.
- Allows the swapper hot key to:
 - `setSwapper` — rotate the swapper role (swapper-gated).
 - `swapGas` — convert `accruedUsdcReimbursements` USDC into WETH via the Universal Router.
 - `withdrawReservedUsdc` — escape-hatch withdrawal of reserved USDC, bounded by the reservation counter.
- Allows users to:
 - `initialize` — one-time proxy initialisation setting `projectOwner`, seeding swapper, and the default `minBridgeAmount` / `twapPeriod` / `bridgeConfirmations`.
 - `bridge` — lock-or-burn USDC and send a cross-chain message to the fixed destination chain, collecting any project fee and

forwarding the remaining msg.value to the gateway.	
---	--



Code Quality

This audit scope involves the smart contracts of the VIA Labs project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the VIA Labs project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] ViaGatewayV1#process - Unsigned gasRefundAmount can drain the bridge WETH treasury

Description

ViaGatewayV1.process() accepts gasRefundAmount from the relayer as calldata, but the value is not included in the signed message hash. BridgeManagerVG1 also does not set a default maxGas cap during initialization, while ViaIntegrationV1.setMessageGateway() grants the gas collector a maximum WETH allowance. A relayer can therefore submit a valid signed message with an inflated gasRefundAmount and pull WETH from the bridge.

Vulnerability Details

ViaGatewayV1.process() accepts gasRefundAmount at lib/vg1-evm-contracts/src/ViaGatewayV1.sol:413-423. The signed hash at lib/vg1-evm-contracts/src/ViaGatewayV1.sol:629-639 includes txId, chain IDs, sender, recipient, onChainData, and offChainData, but excludes gasRefundAmount. The refund is executed at lib/vg1-evm-contracts/src/ViaGatewayV1.sol:447-448 before the bridge callback.

GasRefundV1.refund() transfers the gas token from the recipient project contract to the relayer at lib/vg1-evm-contracts/src/GasRefundV1.sol:119-147. Its cap is skipped when maxGas[recipient] == 0 at lib/vg1-evm-contracts/src/GasRefundV1.sol:127-134, and BridgeManagerVG1.initialize() does not set maxGas at src/BridgeManagerVG1.sol:226-246. The bridge has already max-approved the gas collector through ViaIntegrationV1.setMessageGateway() at lib/vg1-evm-contracts/src/ViaIntegrationV1.sol:198-205.

Proof of Concept

```
function testUnsignedGasRefundDrainsWeth() external {
    bytes[] memory signatures = signedMessageWithoutGasRefundAmount();
    uint256 amountToSteal = weth.balanceOf(address(bridge));

    gateway.process(
        txId,
        sourceChainId,
        destinationChainId,
        sourceEndpoint,
        bytes32(uint256(uint160(address(bridge)))),
        onChainData,
        offChainData,
        signatures,
        amountToSteal
    );

    assertEq(weth.balanceOf(relayer), amountToSteal);
    assertEq(weth.balanceOf(address(bridge)), 0);
}
```

Impact

The bridge's WETH operational treasury can be drained up to the bridge WETH balance, allowance, and any configured `maxGas`. Under the default uncapped state, this is direct treasury fund loss.

Recommendation

Include `gasRefundAmount` or a committed maximum refund in the signed message. Set a nonzero `maxGas` before enabling the gateway, and treat `maxGas == 0` as disabled rather than unlimited.

Note

This finding relates to gateway components that were not part of the original dedicated audit scope. The issue was identified during a limited review of the interaction between the in scope bridge contract and the surrounding gateway flow. Given that the protocol

informed Hashlock they will be using their own set of relayers, we are closing this issue with the above risks in mind.

Status

Resolved



[H-02] ViaGatewayV1#process - Relay-controlled gas refunds can both retry failed messages and reduce recipient delivery

Description

The same relay-controlled `gasRefundAmount` affects two different paths. The gateway pays the WETH refund before executing the bridge callback and only marks the message processed after success, so failed callbacks remain retryable after WETH has already been paid. In dynamic mode, `BridgeManagerVG1.messageProcess()` also converts that value into a USDC reimbursement and deducts it from the recipient's bridged amount, with only `usdcNeeded < amount` as the bridge-level bound.

Vulnerability Details

`ViaGatewayV1.process()` checks `processedMessages[txId]` at `lib/vg1-evm-contracts/src/ViaGatewayV1.sol:424-427`, pays the refund at `lib/vg1-evm-contracts/src/ViaGatewayV1.sol:447-448`, and only then executes the callback through `processCall()` at `lib/vg1-evm-contracts/src/ViaGatewayV1.sol:452-463`. `processedMessages[txId]` is set only on callback success at `lib/vg1-evm-contracts/src/ViaGatewayV1.sol:464-467`; callback failures are caught and emitted without marking the message processed at `lib/vg1-evm-contracts/src/ViaGatewayV1.sol:468-475`.

On the bridge side, when `swapEnabled` is true, `messageProcess()` quotes the same `gasRefundAmount` through `_quoteUsdcForWeth()` at `src/BridgeManagerVG1.sol:329-332`. It rejects only when `usdcNeeded >= amount` at `src/BridgeManagerVG1.sol:334-337`. Accepted reimbursement is added to `accruedUsdcReimbursements` at `src/BridgeManagerVG1.sol:339-341`, and the recipient receives only `amount - reimbursement` at `src/BridgeManagerVG1.sol:348-361`. The quote path uses `gasRefundAmount` at `src/BridgeManagerVG1.sol:375-391`, while the signed gateway hash excludes `gasRefundAmount` at `lib/vg1-evm-contracts/src/ViaGatewayV1.sol:629-639`.

Proof of Concept

```
function testRelayerRefundCanRetryAndConsumeRecipientAmount() external {
```

```

uint256 amount = 1_000e6;
uint256 retryRefund = refundThatForcesAmountBelowGasFee(amount);
uint256 deliveryRefund = refundThatQuotesTo(amount - 1);
bytes memory message = abi.encode(bytes32(uint256(uint160(alice))), amount);

gateway.process(txId, srcChain, dstChain, sourceEndpoint, bridgeEndpoint,
message, "", signatures, retryRefund);
assertEq(weth.balanceOf(relayer), retryRefund);
assertFalse(gateway.processedMessages(txId));

gateway.process(txId, srcChain, dstChain, sourceEndpoint, bridgeEndpoint,
message, "", signatures, retryRefund);
assertEq(weth.balanceOf(relayer), retryRefund * 2);
assertFalse(gateway.processedMessages(txId));

gateway.process(secondTxId, srcChain, dstChain, sourceEndpoint, bridgeEndpoint,
message, "", signatures, deliveryRefund);
assertEq(usdc.balanceOf(alice), 1);
assertEq(bridge.accruedUsdcReimbursements(), amount - 1);
}

```

Impact

Failed messages can repeatedly pay WETH refunds because the refund happens before successful finalization. Separately, successfully delivered dynamic-mode messages can leave recipients with nearly zero USDC while diverting the rest into the reimbursement reserve. A finite `maxGas` caps each attempt, but does not prevent repeated failed attempts or a per-message recipient deduction up to `amount - 1`.

Recommendation

Pay gas refunds only after successful callback execution and message finalization. Include a signed maximum refund or signed recipient minimum amount in the message, and enforce an explicit bridge-level cap on the USDC reimbursement deducted from recipients.

Note

This finding relates to gateway components that were not part of the original dedicated audit scope. The issue was identified during a limited review of the interaction between the in scope bridge contract and the surrounding gateway flow. Given that the protocol informed Hashlock they will be using their own set of relayers, we are closing this issue with the above risks in mind.

Status

Resolved

Medium

[M-01] BridgeManagerVG1#bridge - Single-confirmation default exposes bridge messages to source-chain reorg double spending

Description

BridgeManagerVG1 initializes bridgeConfirmations to 1 and forwards that value to ViaGatewayV1.send() for every bridge message. The gateway does not enforce finality on-chain; it only emits the confirmations value as part of SendRequested. If relayers act on this one-confirmation hint, a short source-chain reorg can remove the lock or burn transaction after the destination message has already been processed, creating a cross-chain double-spend condition.

Vulnerability Details

initialize() sets bridgeConfirmations = 1 at src/BridgeManagerVG1.sol:243-245. bridge() forwards uint16(bridgeConfirmations) into IViaGatewayV1.send() at src/BridgeManagerVG1.sol:286-292. ViaGatewayV1.send() receives the confirmations argument at lib/vg1-evm-contracts/src/ViaGatewayV1.sol:353-358, but it only emits the value in SendRequested at lib/vg1-evm-contracts/src/ViaGatewayV1.sol:370-377; there is no on-chain finality check.

The setter comment describes the value as a block-confirmations hint and explicitly allows 0 as a relayer-default hint at src/BridgeManagerVG1.sol:452-456. This means finality enforcement is delegated to off-chain relayers and signers. A one-block default is unsafe for chains where normal or adversarial reorg depth can exceed one block.

Proof of Concept

```
function testOneConfirmationReorgDoubleSpendScenario() external {
    assertEq(bridge.bridgeConfirmations(), 1);

    vm.prank(user);
    uint256 txId = bridge.bridge(bytes32(uint256(uint160(destinationUser))),
```

```
1_000e6);

    // Off-chain relayers observe SendRequested(txId, ..., confirmations = 1)
    // and process the destination message after one source confirmation.
    processDestinationMessage(txId);

    // A source-chain reorg deeper than one block removes the original bridge()
    // transaction, so the source-side lock or burn no longer exists while the
    // destination-side release or mint already happened.
    simulateSourceReorgRemovingBridgeTransaction(txId);
}
```

Impact

The destination chain can release or mint funds for a source transaction that is later removed by a source-chain reorg. The sender can retain or recover the source-side funds while still receiving value on the destination chain.

Recommendation

Set `bridgeConfirmations` to a chain-specific minimum finality threshold rather than 1. Maintain a per-chain confirmation policy keyed by source chain ID, and require relayers/signers to wait for that threshold before signing or processing destination messages.

Status

Resolved

[M-02] BridgeManagerVG1#bridge - Gateway source fees are paid from bridge liquidity when source-gas mode is disabled

Description

BridgeManagerVG1.bridge() pulls exactly the user-specified amount and encodes that same gross amount for destination delivery. The gateway then charges the fee collector with msg.sender == bridge, so FeeCollector token fees are paid from the bridge's token balance rather than from a separately collected user fee. The problematic path is especially direct when FeeCollectorV1.pay() computes isGasSourceToo == false for the bridge, because no native source-gas fee is required from the bridge call and the token fee is still pulled from bridge liquidity.

Vulnerability Details

bridge() transfers amount from the user at src/BridgeManagerVG1.sol:257-272 and encodes the same amount into onChainData at src/BridgeManagerVG1.sol:279-292. ViaGatewayV1.send() then calls feeCollector.pay(msg.sender) at lib/vg1-evm-contracts/src/ViaGatewayV1.sol:353-363, where msg.sender is the bridge.

Inside FeeCollectorV1.pay(), isGasSourceToo is read from sourceGasAmountWhitelist[sender] at lib/vg1-evm-contracts/src/FeeCollectorV1.sol:140-144. If isGasSourceToo == false for sender == address(BridgeManagerVG1), the exact native source-gas check at lib/vg1-evm-contracts/src/FeeCollectorV1.sol:146-153 is skipped. The function still computes the token fee and transfers feeToken from sender to beneficiary at lib/vg1-evm-contracts/src/FeeCollectorV1.sol:155-164.

The default feeAmount is 250000 at lib/vg1-evm-contracts/src/FeeCollectorV1.sol:36-42, and maxFee is only enforced when nonzero at lib/vg1-evm-contracts/src/FeeCollectorV1.sol:140-158. ViaIntegrationV1.setMessageGateway() grants the fee collector maximum allowance at lib/vg1-evm-contracts/src/ViaIntegrationV1.sol:208-216. The existing fork test confirms the bridge balance increases by amount - 250000 while the beneficiary receives 250000 at test/fork/BridgeManagerVG1Native.fork.t.sol:76-88.



Proof of Concept

```
function testBridgeFeeComesFromLockedLiquidityWhenSourceGasDisabled() external {
    uint256 amount = 100e6;
    uint256 beforeBridge = usdc.balanceOf(address(bridge));
    uint256 beforeBeneficiary = usdc.balanceOf(beneficiary);

    assertFalse(feeCollector.sourceGasAmountWhitelist(address(bridge)));

    vm.prank(user);
    bridge.bridge(bytes32(uint256(uint160(destinationUser))), amount);

    assertEq(usdc.balanceOf(address(bridge)), beforeBridge + amount - 250_000);
    assertEq(usdc.balanceOf(beneficiary), beforeBeneficiary + 250_000);
}
```

Impact

Native-side locked liquidity can become undercollateralized relative to the gross destination amount when the gateway token fee is pulled from the bridge balance. A fee increase can also cause bridging denial of service or drain existing bridge balances through normal user bridge calls.

Recommendation

Set a nonzero `maxFee` before enabling sends. Charge gateway token fees explicitly from users or maintain a segregated fee reserve that cannot consume locked bridge liquidity. If `sourceGasAmountWhitelist[address(bridge)]` is expected to be false, document that no native source-gas payment protects this path and ensure the token fee cannot be taken from locked backing.

Status

Resolved

[M-03] GasRefundV1#refund - Whitelist overrides desynchronize WETH paid from USDC charged

Description

GasRefundV1 can replace raw gasRefundAmount with a whitelist-derived effective refund, but BridgeManagerVG1.messageProcess() still accounts using the raw relayer-supplied value. When a fixed or free refund override is configured for the bridge, the WETH actually paid and the USDC deducted from the recipient can diverge.

Vulnerability Details

ViaGatewayV1.process() passes the same raw gasRefundAmount to gasCollector.refund() and later to the bridge callback at lib/vg1-evm-contracts/src/ViaGatewayV1.sol:447-463. GasRefundV1.refund() computes the effective refund through _getAmount() at lib/vg1-evm-contracts/src/GasRefundV1.sol:127-134, while _getAmount() can return 0 for FREE or a fixed whitelist amount at lib/vg1-evm-contracts/src/GasRefundV1.sol:189-202.

BridgeManagerVG1.messageProcess() does not receive the effective refund. It quotes the raw value into USDC at src/BridgeManagerVG1.sol:329-332 and increments accruedUsdcReimbursements at src/BridgeManagerVG1.sol:339-341.

Proof of Concept

```
function testWhitelistFixedRefundUnderchargesBridgeAccounting() external {
    gasRefund.setWhitelist(address(bridge), 0.1 ether);

    uint256 rawRefund = 1;

    gateway.process(
        txId,
        srcChain,
        dstChain,
        sourceEndpoint,
        bridgeEndpoint,
```

```
onChainData,  
    "",  
    signatures,  
    rawRefund  
);  
  
assertEq(weth.balanceOf(relayer), 0.1 ether);  
assertLt(bridge.accruedUsdcReimbursements(), quoteUsdc(0.1 ether));  
}
```

Impact

If the whitelist amount is higher than the raw value, the bridge can pay more WETH than it charges in USDC. If the whitelist amount is lower or FREE, users can be charged USDC for WETH that was not actually paid. The issue is conditional on dynamic mode and a non-default gas-refund whitelist entry for the bridge.

Recommendation

Make `BridgeManagerVG1` account against the same effective refund amount used by `GasRefundV1`. Alternatively, disable whitelist overrides for bridge recipients or include the effective refund in the signed and callback data.

Status

Resolved

Low

[L-01] BridgeManagerVG1#withdrawReservedUsdc - Swapper hot key can drain reserved USDC and cannot be evicted by the owner

Description

The `swapper` address is intended to be an operational hot key, but it can withdraw reserved USDC to any address and is the only role allowed to rotate itself. If the swapper key is compromised, the owner cannot rotate or revoke it through the normal role path.

Vulnerability Details

The `onlySwapper` modifier only checks `msg.sender == swapper` at `src/BridgeManagerVG1.sol:188-192`. `setSwapper()` is also `onlySwapper` and has no owner override at `src/BridgeManagerVG1.sol:462-471`. `withdrawReservedUsdc()` lets the swapper transfer `accruedUsdcReimbursements` to any nonzero address at `src/BridgeManagerVG1.sol:685-697`.

The test suite confirms this trust inversion: after rotation, the owner cannot call `setSwapper()` and the call reverts at `test/unit/BridgeManagerVG1.t.sol:868-876`.

Proof of Concept

```
function testCompromisedSwapperDrainsReserveAndLocksRecovery() external {
    vm.prank(address(bridge.swapper()));
    bridge.setSwapper(attacker);

    uint256 reserve = bridge.accruedUsdcReimbursements();

    vm.prank(attacker);
    bridge.withdrawReservedUsdc(attacker, reserve);

    vm.prank(attacker);
    bridge.setSwapper(address(0));
```

```
vm.expectRevert(BridgeManagerVG1.NotSwapper.selector);  
bridge.setSwapper(ownerRecoveryKey);  
}
```

Impact

Reserved USDC can be stolen by a compromised swapper, and normal owner recovery is unavailable. The loss is bounded by `accruedUsdcReimbursements`.

Recommendation

Make swapper rotation owner-controlled or add an owner emergency revoke path. Split bot execution rights from arbitrary reserve withdrawal rights.

Status

Acknowledged

[L-02] `ViaIntegrationV1#setMessageGateway` - Gateway rotation leaves stale gas collector allowances

Description

`ViaIntegrationV1.setMessageGateway()` grants the new gas collector maximum allowance but never revokes allowance from the old gas collector. After gateway rotation, an old gateway and old gas collector can still use the stale WETH allowance to pull refunds before the bridge rejects the callback from the old gateway.

Vulnerability Details

`setMessageGateway()` overwrites `gateway`, `gasCollector`, and `feeCollector`, then `max-approves` the new `gasCollector` at `lib/vg1-evm-contracts/src/ViaIntegrationV1.sol:192-205`. There is no revocation of the previously approved collector.

An old `GasRefundV1` instance can still be called by its configured old gateway at `lib/vg1-evm-contracts/src/GasRefundV1.sol:119-124`. The old gateway pays the refund at `lib/vg1-evm-contracts/src/ViaGatewayV1.sol:447-448`. The bridge callback then fails because `ViaIntegrationV1.messageProcessFromGateway()` requires `msg.sender == gateway` at `lib/vg1-evm-contracts/src/ViaIntegrationV1.sol:52-60` and the bridge now stores the new gateway. Since the old gateway marks messages processed only on callback success at `lib/vg1-evm-contracts/src/ViaGatewayV1.sol:464-467`, the stale path can be retried while old validation and allowance remain usable.

Proof of Concept

```
function testOldGatewayCanUseStaleGasCollectorAllowance() external {
    address oldGateway = address(gateway);
    address oldGasCollector = address(gasCollector);

    projectOwnerCallsSetMessageGateway(newGateway);

    assertGt(weth.allowance(address(bridge), oldGasCollector), 0);
}
```

```

vm.prank(relayer);
ViaGatewayV1(oldGateway).process(
    txId,
    srcChain,
    dstChain,
    sourceEndpoint,
    bridgeEndpoint,
    onChainData,
    "",
    signatures,
    0.1 ether
);

assertEq(weth.balanceOf(relayer), 0.1 ether);
assertFalse(ViaGatewayV1(oldGateway).processedMessages(txId));
}

```

Impact

Gateway rotation does not fully deauthorize retired infrastructure. A retired gateway path can extract WETH from the bridge if the old gateway remains enabled and can validate messages.

Recommendation

Revoke old gas collector allowance before approving the new collector. Add explicit allowance-revocation helpers and require rotation runbooks to verify zero allowance for retired collectors.

Status

Resolved

[L-03] BridgeManagerVG1#setSwapper - Zero-address swapper can brick swapper-only operations

Description

`setSwapper()` accepts `address(0)`. Because the function itself is gated by `onlySwapper`, setting the swapper to zero makes future calls to `swapGas()`, `withdrawReservedUsdc()`, and `setSwapper()` impossible through the normal role path.

Vulnerability Details

`setSwapper()` has no zero-address rejection at `src/BridgeManagerVG1.sol:462-471`. The comment states that setting `address(0)` revokes the role and is irreversible without an upgrade at `src/BridgeManagerVG1.sol:462-466`. `swapGas()` and `withdrawReservedUsdc()` are both `onlySwapper` at `src/BridgeManagerVG1.sol:645-648` and `src/BridgeManagerVG1.sol:685-688`.

Proof of Concept

```
function testZeroSwapperBricksOperations() external {
    vm.prank(bridge.swapper());
    bridge.setSwapper(address(0));

    vm.expectRevert(BridgeManagerVG1.NotSwapper.selector);
    bridge.swapGas(1, block.timestamp + 1);

    vm.expectRevert(BridgeManagerVG1.NotSwapper.selector);
    bridge.setSwapper(recoverySwapper);
}
```

Impact

The USDC reimbursement reserve and WETH swap workflow can become unavailable until a UUPS upgrade or equivalent privileged recovery.

Recommendation

Reject `address(0)` in `setSwapper()` and provide an explicit owner-controlled revoke or emergency rotation mechanism.

Status

Resolved



[L-04] BridgeManagerVG1#swapGas - Swap execution lacks a bridge-enforced pricing sanity floor

Description

The bridge charges users through a TWAP quote but does not enforce a corresponding on-chain output floor when swapping reserved USDC into WETH. It also allows the quote pool and execution fee tier to diverge and does not validate the token identity of `swapPool`.

Vulnerability Details

User reimbursements are priced through `_quoteUsdcForWeth()` using `swapPool` at `src/BridgeManagerVG1.sol:375-391`. `swapGas()` relies entirely on caller-supplied `minAmountOut` at `src/BridgeManagerVG1.sol:645-674`. The actual Universal Router path is built from `FIAT_TOKEN`, `swapPoolFee`, and `weth` at `src/BridgeManagerVG1.sol:666-671`, so the execution venue can differ from the pool used for accounting.

`setSwapPool()` only rejects zero and does not validate pool tokens at `src/BridgeManagerVG1.sol:499-503`. `setSwapSlippageBps()` only enforces a lower bound at `src/BridgeManagerVG1.sol:509-513`. `setSwapEnabled(true)` only checks nonzero configuration and lower-bound slippage at `src/BridgeManagerVG1.sol:534-544`.

Proof of Concept

```
function testWeakMinOutAcceptsBadExecutionPrice() external {
    bridge.setSwapPool(deepPoolUsedForQuote);
    bridge.setSwapConfig(universalRouter, address(weth),
    thinPoolFeeUsedForExecution);
    bridge.setSwapSlippageBps(10_000);
    bridge.setSwapEnabled(true);

    accrueUsdcReimbursements(10_000e6);

    vm.prank(swapper);
    bridge.swapGas(1, block.timestamp + 60);
}
```

```
assertLt(weth.balanceOf(address(bridge)), expectedTwapWethOut);  
}
```

Impact

Reserved USDC can be swapped for materially less WETH than the accounting quote implies. Bad configuration can also cause delivery reverts or leave the swap path operationally inconsistent.

Recommendation

Validate that `swapPool` is a `FIAT_TOKEN/weth` Uniswap V3 pool and enforce consistency between `swapPool` and `swapPoolFee`. Derive a minimum output from the same TWAP accounting model used to charge users.

Status

Resolved

[L-05] BridgeManagerVG1#bridge - Zero-amount messages are possible when minBridgeAmount is zero

Description

The source-side `bridge()` function does not independently reject `amount == 0`. If the owner sets `minBridgeAmount` to zero, users can emit bridge messages that always fail on the destination side.

Vulnerability Details

`bridge()` checks only `amount < minBridgeAmount` at `src/BridgeManagerVG1.sol:257-263`. `setMinBridgeAmount()` permits any value, including zero, at `src/BridgeManagerVG1.sol:446-450`. The source then encodes the unchecked `amount` into `onChainData` at `src/BridgeManagerVG1.sol:279-292`.

On the destination side, `messageProcess()` rejects `amount == 0` at `src/BridgeManagerVG1.sol:317-323`. Because the gateway refund is paid before the callback and failed messages remain unprocessed, this can compose with the refund replay issue.

Proof of Concept

```
function testZeroAmountBridgeCreatesUndeliverableMessage() external {
    bridge.setMinBridgeAmount(0);

    vm.prank(user);
    uint256 txId = bridge.bridge(bytes32(uint256(uint160(user))), 0);

    gateway.process(txId, srcChain, dstChain, sourceEndpoint, bridgeEndpoint,
abi.encode(bytes32(uint256(uint160(user))), 0), "", signatures, 1);

    assertFalse(gateway.processedMessages(txId));
}
```

Impact

Zero-amount messages can grief relayers and create permanently failing cross-chain messages. With a positive refund, the same message can also trigger repeated WETH refunds under the gateway ordering bug.

Recommendation

Reject `amount == 0` in `bridge()` regardless of the configured minimum bridge amount.

Status

Resolved

[L-06] BridgeManagerVG1#bridge - Excess native value is forwarded and can become unrecoverable

Description

When native project fees are enabled, `_collectProjectFee()` consumes only `projectFeeAmount` and `bridge()` forwards the rest of `msg.value` to the gateway. If the downstream `FeeCollectorV1` does not consume that value, it has no native rescue path.

Vulnerability Details

`_collectProjectFee()` accepts `msg.value >= projectFeeAmount` and returns only the consumed project fee at `src/BridgeManagerVG1.sol:405-416`. `bridge()` forwards `msg.value - nativeProjectFee` to `ViaGatewayV1.send()` at `src/BridgeManagerVG1.sol:286-292`. The gateway forwards that native value to `FeeCollectorV1.pay()` at `lib/vg1-evm-contracts/src/ViaGatewayV1.sol:353-363`.

`FeeCollectorV1.pay()` only uses native value when `sourceGasAmountWhitelist[sender]` is enabled, and then it requires exact equality with `sourceGasAmount` at `lib/vg1-evm-contracts/src/FeeCollectorV1.sol:143-153`. `FeeCollectorV1` has no native withdraw or sweep function; its only native transfer out is the exact source-gas forwarding call at `lib/vg1-evm-contracts/src/FeeCollectorV1.sol:137-165`. The existing unit test confirms excess native is forwarded away from the bridge at `test/unit/BridgeManagerVG1.t.sol:485-503`.

Proof of Concept

```
function testExcessNativeValueLeavesBridgeAndCannotBeRecovered() external {
    bridge.setProjectFee(address(0), 0.01 ether);

    vm.deal(user, 1 ether);
    vm.prank(user);
    bridge.bridge{value: 0.02 ether}(bytes32(uint256(uint160(user))), 50e6);

    assertEq(bridge.accruedProjectFee(), 0.01 ether);
    assertEq(address(feeCollector).balance, 0.01 ether);
}
```

```
}
```

Impact

Users can permanently lose excess native currency. Source-gas-whitelisted configurations are also brittle because the forwarded value must exactly match the downstream source gas amount.

Recommendation

Require exact native value for the active fee mode, refund any excess, or add a controlled native rescue path to FeeCollectorV1.

Status

Resolved

[L-07] BridgeManagerVG1#bridge - Noncanonical recipient bytes can misdeliver or create failing messages

Description

The source-side bridge accepts any nonzero `bytes32` recipient. The destination side converts that value to an EVM address by truncating to the low 160 bits and rejects only the resulting zero address. Nonzero high 96 bits are never rejected.

Vulnerability Details

Users control `to` in `bridge(bytes32 to, uint256 amount)`, and the source only rejects `bytes32(0)` at `src/BridgeManagerVG1.sol:257-263`. The value is encoded into `onChainData` at `src/BridgeManagerVG1.sol:279`.

On destination, `messageProcess()` decodes the same `bytes32` at `src/BridgeManagerVG1.sol:317-320` and calls `_bytes32ToAddress()` at `src/BridgeManagerVG1.sol:324`. The inherited helper truncates to `address(uint160(uint256(addr)))` at `lib/vg1-evm-contracts/src/ViaIntegrationV1.sol:332-333`. Only `address(0)` is rejected at `src/BridgeManagerVG1.sol:324-325`.

Proof of Concept

```
function testNoncanonicalRecipientTruncatesToDifferentAddress() external {
    bytes32 noncanonical = bytes32(uint256(1) << 160) |
    bytes32(uint256(uint160(victim)));

    vm.prank(user);
    bridge.bridge(noncanonical, 100e6);

    processInboundMessage(abi.encode(noncanonical, 100e6), 0);

    assertEq(usdc.balanceOf(victim), 100e6);
}
```

Impact

Recipient bytes shown in events and off-chain systems can differ from the actual payout address. If the low 160 bits are zero, the signed message becomes permanently failing and can compose with the refund replay issue.

Recommendation

Require canonical EVM address encoding by rejecting any recipient value with nonzero high 96 bits. For non-EVM destinations, use a destination-specific encoding path instead of silently truncating.

Status

Resolved

[L-08] BridgeManagerVG1#_collectProjectFee - Fee-on-transfer project fee tokens overstate accrued fees

Description

`_collectProjectFee()` assumes that the contract receives exactly `projectFeeAmount` when an ERC-20 project fee is collected. Fee-on-transfer or deflationary tokens can transfer less than the requested amount, while the bridge still increments `accruedProjectFee` by the nominal amount.

Vulnerability Details

The audit tag at `src/BridgeManagerVG1.sol:405` flags the exact issue. In the ERC-20 fee branch, `_collectProjectFee()` calls `safeTransferFrom()` for `amount` at `src/BridgeManagerVG1.sol:418-423`, but it does not compare the contract's token balance before and after the transfer. It then increments `accruedProjectFee` by the nominal amount at `src/BridgeManagerVG1.sol:425`.

`withdrawProjectFees()` later withdraws the full `accruedProjectFee` amount at `src/BridgeManagerVG1.sol:612-632`. If the fee token charged a transfer fee on collection, the accrued counter can exceed the token balance actually received by the bridge.

Proof of Concept

```
function testFeeOnTransferProjectFeeOverAccrues() external {
    feeOnTransferToken.setTransferTaxBps(1_000);
    bridge.setProjectFee(address(feeOnTransferToken), 100e18);

    vm.prank(user);
    bridge.bridge(bytes32(uint256(uint160(destinationUser))), 1_000e6);

    assertEq(bridge.accruedProjectFee(), 100e18);
    assertEq(feeOnTransferToken.balanceOf(address(bridge)), 90e18);

    vm.expectRevert();
    bridge.withdrawProjectFees(projectTreasury);
}
```

```
}
```

Impact

Project-fee accounting can become insolvent for fee-on-transfer tokens. Withdrawals can revert, or if the bridge holds unrelated balances of the same token, withdrawals can consume unrelated funds to satisfy the overstated fee counter.

Recommendation

Do not support fee-on-transfer project fee tokens, or account for the actual balance delta received by the bridge. If such tokens are intentionally supported, accrue only the received amount and document the outgoing transfer-fee behavior.

Status

Resolved

QA

[Q-01] BridgeManagerVG1 - Force-fed or stray native ETH is unrecoverable

Description

BridgeManagerVG1 has no general native ETH rescue function. Non-accrued native ETH can be forced into the contract and cannot be withdrawn through the existing project-fee or WETH withdrawal paths.

Vulnerability Details

BridgeManagerVG1 has no `receive()` or `fallback()` function. `withdrawProjectFees()` can withdraw native ETH only up to `accruedProjectFee` at `src/BridgeManagerVG1.sol:612-632`. `withdrawWeth()` withdraws WETH only at `src/BridgeManagerVG1.sol:700-708`.

Impact

Native ETH can become permanently stranded until a UUPS upgrade adds a recovery path. There is no direct attacker profit, but treasury accounting and recovery are impaired.

Recommendation

Add a narrowly scoped native rescue function or explicitly document that non-accrued native ETH is intentionally unrecoverable until upgrade.

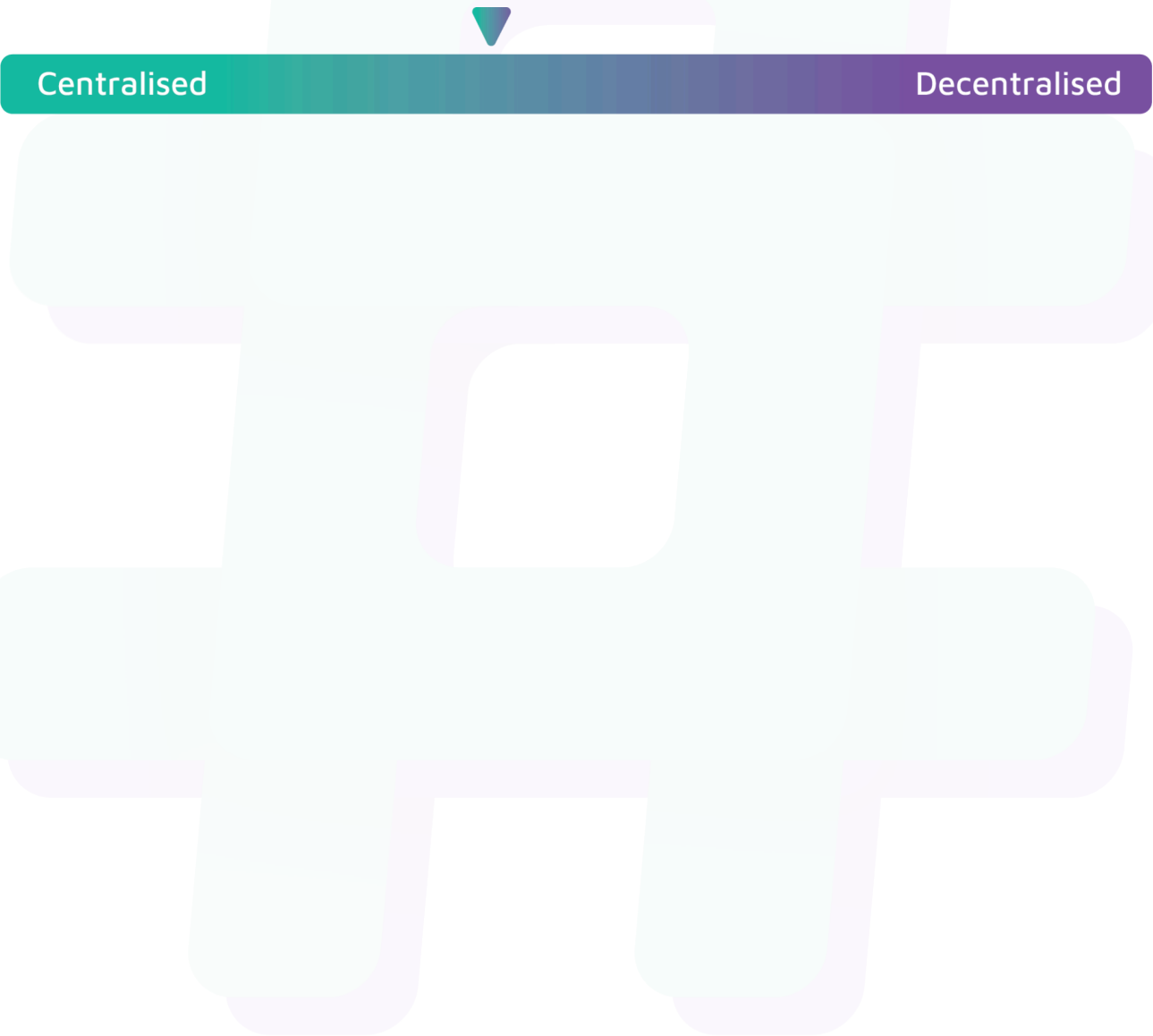
Status

Resolved

Centralisation

The VIA Labs project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

To the best of our ability, Hashlock is not able to identify any further vulnerabilities within the agreed audit scope. This conclusion does not extend to contracts, integrations, gateway components, relayer infrastructure, or external systems outside the listed audit scope, which may contain additional risks or vulnerabilities if not reviewed under a dedicated scope.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.